Valentin LE GALL
Elia LEPREVOST

Bases de données réparties

Nano-Satellites

PHASE 2
Schéma Oracle & Triggers
Projet

ALTN83-2526BSP01 - Bases de données réparties
Christophe CROISANT

Partie 1 — Création du schéma (DDL) **3**
Partie 2 — Jeu de données initial (DML) **11**
Partie 3 — Les 5 triggers métier NanoOrbit **20**
Partie 4 — Script de contrôle **21**

Partie 1 — Création du schéma (DDL)

Dans cette partie, nous allons écrire le DDL afin de créer les tables dans la base de données.

Pour chaque table, nous allons mettre le script, ainsi que l'explication des contraintes et indexations.

ORBITE

```
CREATE TABLE ORBITE(
   id_orbite VARCHAR2(20),
   type_orbite VARCHAR2(10) NOT NULL,
   altitude NUMBER(5) NOT NULL,
   inclinaison NUMBER(5,2) NOT NULL,
   periode_orbitale NUMBER(6,2) NOT NULL,
   excentricite NUMBER(6,4) NOT NULL,
   zone_couverture VARCHAR2(200),
   CONSTRAINT pk_orbite PRIMARY KEY(id_orbite),
   CONSTRAINT uq_orbite_alt_inc UNIQUE(altitude, inclinaison),
   CONSTRAINT ck_orbite_type CHECK (type_orbite IN ('LEO', 'MEO', 'SSO',
'GEO')),
   CONSTRAINT ck_orbite_altitude CHECK (altitude > 0),
   CONSTRAINT ck_orbite_inclinaison CHECK (inclinaison BETWEEN 0 AND
180),
   CONSTRAINT ck_orbite_periode CHECK (periode_orbitale > 0),
   CONSTRAINT ck_orbite_excentricite CHECK (excentricite >= 0 AND
excentricite < 1)
);
```

La clé primaire pk_orbite identifie chaque orbite de manière unique. La contrainte uq_orbite_alt_inc répond directement à la règle demandée dans le sujet : le couple (altitude, inclinaison) doit être unique. Les CHECK sur type_orbite, altitude, inclinaison, periode_orbitale et excentricite empêchent des valeurs incohérentes physiquement.

Côté indexation, la clé primaire crée déjà un index sur id_orbite, et la contrainte UNIQUE crée aussi un index sur (altitude, inclinaison). Il n'est donc pas nécessaire d'ajouter d'autres index sur cette table, car les recherches principales se feront soit par identifiant, soit par couple orbital.

SATELLITE

```
CREATE TABLE SATELLITE(
   id_satellite VARCHAR2(20),
   nom_satellite VARCHAR2(100) NOT NULL,
   date_lancement DATE NOT NULL,
```

```
   masse NUMBER(5,2) NOT NULL,
   format_cubesat VARCHAR2(5) NOT NULL,
   statut VARCHAR2(30) NOT NULL,
   duree_vie_prevue NUMBER(5) NOT NULL,
   capacite_batterie NUMBER(6,1) NOT NULL,
   id_orbite VARCHAR2(20) NOT NULL,
   CONSTRAINT pk_satellite PRIMARY KEY(id_satellite),
   CONSTRAINT fk_satellite_orbite FOREIGN KEY(id_orbite) REFERENCES
ORBITE(id_orbite),
   CONSTRAINT uq_satellite_nom UNIQUE(nom_satellite),
   CONSTRAINT ck_satellite_masse CHECK (masse > 0),
   CONSTRAINT ck_satellite_format CHECK (format_cubesat IN ('1U', '3U',
'6U', '12U')),
   CONSTRAINT ck_satellite_statut CHECK (statut IN ('Opérationnel', 'En
veille', 'Défaillant', 'Désorbité')),
   CONSTRAINT ck_satellite_duree CHECK (duree_vie_prevue > 0),
   CONSTRAINT ck_satellite_batterie CHECK (capacite_batterie > 0)
);

CREATE INDEX idx_satellite_id_orbite
   ON SATELLITE(id_orbite);

CREATE INDEX idx_satellite_statut
   ON SATELLITE(statut);
```

La clé primaire pk_satellite identifie chaque satellite. La clé étrangère fk_satellite_orbite relie chaque satellite à une orbite, ce qui correspond bien à la règle métier selon laquelle un satellite est associé à une seule orbite. Les CHECK encadrent les formats CubeSat, les statuts, ainsi que les valeurs numériques strictement positives. Le UNIQUE sur nom_satellite n'était pas demandé mais il semble logique que deux satellites ne puissent pas partager de nom.

Côté indexation, la PK indexe id_satellite. L'index idx_satellite_id_orbite est important pour accélérer les jointures avec ORBITE. L'index idx_satellite_statut est utile pour les requêtes opérationnelles du type "afficher les satellites désorbitées / en veille / défaillants".

INSTRUMENT

```
CREATE TABLE INSTRUMENT(
   ref_instrument VARCHAR2(20),
   type_instrument VARCHAR2(50) NOT NULL,
   modele VARCHAR2(100) NOT NULL,
   resolution NUMBER(6,1),
   consommation NUMBER(5,2) NOT NULL,
   masse NUMBER(5,3) NOT NULL,
```

```
   CONSTRAINT pk_instrument PRIMARY KEY(ref_instrument),
   CONSTRAINT ck_instrument_resolution CHECK (resolution IS NULL OR
resolution > 0),
   CONSTRAINT ck_instrument_consommation CHECK (consommation > 0),
   CONSTRAINT ck_instrument_masse CHECK (masse > 0)
);
```

La clé primaire pk_instrument garantit l'unicité de la référence instrument. Les contraintes CHECK imposent une résolution toujours positive lorsqu'elle est renseignée, et des valeurs strictement positives pour la consommation et la masse. Cela respecte bien le fait que certains instruments, comme AIS, peuvent avoir une résolution nulle au sens métier donc stockée à NULL.

Côté indexation, la PK suffit dans la plupart des cas. Il n'y a pas besoin d'ajouter d'index supplémentaire tant que les recherches se font surtout par ref_instrument. Les filtres sur type_instrument ou modele peuvent exister, mais ils ne sont pas les plus critiques dans le sujet.

CENTRE_CONTROLE

```
CREATE TABLE CENTRE_CONTROLE(
   id_centre VARCHAR2(20),
   nom_centre VARCHAR2(100) NOT NULL,
   ville VARCHAR2(50) NOT NULL,
   region_geo VARCHAR2(50) NOT NULL,
   fuseau_horaire VARCHAR2(50) NOT NULL,
   statut VARCHAR2(20) NOT NULL,
   CONSTRAINT pk_centre_controle PRIMARY KEY(id_centre),
   CONSTRAINT ck_centre_region CHECK (region_geo IN ('Europe',
'Amériques', 'Asie-Pacifique')),
   CONSTRAINT ck_centre_statut CHECK (statut IN ('Actif', 'Inactif'))
);
```

La clé primaire pk_centre_controle identifie chaque centre. Les CHECK sur region_geo et statut évitent des valeurs hors domaine. Cette table joue le rôle de référentiel organisationnel.

Côté indexation, l'index de la PK suffit largement car la table a peu de lignes. Ajouter d'autres index serait inutile et alourdirait inutilement les écritures.

STATION_SOL

```
CREATE TABLE STATION_SOL(
   code_station VARCHAR2(20),
   nom_station VARCHAR2(100) NOT NULL,
   latitude NUMBER(9,6) NOT NULL,
```

```
   longitude NUMBER(15,6) NOT NULL,
   diametre_antenne NUMBER(4,1) NOT NULL,
   bande_frequence VARCHAR2(10) NOT NULL,
   debit_max NUMBER(6,1) NOT NULL,
   statut VARCHAR2(20) NOT NULL,
   CONSTRAINT pk_station_sol PRIMARY KEY(code_station),
   CONSTRAINT ck_station_latitude CHECK (latitude BETWEEN -90 AND 90),
   CONSTRAINT ck_station_longitude CHECK (longitude BETWEEN -180 AND
180),
   CONSTRAINT ck_station_diametre CHECK (diametre_antenne > 0),
   CONSTRAINT ck_station_bande CHECK (bande_frequence IN ('UHF', 'S',
'X', 'Ka')),
   CONSTRAINT ck_station_debit CHECK (debit_max > 0),
   CONSTRAINT ck_station_statut CHECK (statut IN ('Active',
'Maintenance', 'Inactive'))
);

CREATE INDEX idx_part_id_mission
   ON PARTICIPATION(id_mission);
```

La clé primaire pk_station_sol identifie chaque station. Les contraintes CHECK encadrent les coordonnées géographiques, le diamètre d'antenne, la bande fréquentielle, le débit maximum et le statut. Cela permet de verrouiller un minimum de cohérence sur les caractéristiques techniques des stations.

Côté indexation, la PK couvre les accès par code_station. L'index idx_station_statut peut être utile pour trouver rapidement les stations actives ou en maintenance, notamment dans les traitements de planification de fenêtres.

MISSION

```
CREATE TABLE MISSION(
   id_mission VARCHAR2(20),
   nom_mission VARCHAR2(100) NOT NULL,
   objectif CLOB NOT NULL,
   zone_geo_cible VARCHAR2(200) NOT NULL,
   date_debut DATE NOT NULL,
   date_fin DATE,
   statut_mission VARCHAR2(20) NOT NULL,
   CONSTRAINT pk_mission PRIMARY KEY(id_mission),
   CONSTRAINT ck_mission_statut CHECK (statut_mission IN ('Active',
'Terminée')),
   CONSTRAINT ck_mission_dates CHECK (date_fin IS NULL OR date_fin >=
date_debut)
);
```

```
CREATE INDEX idx_mission_statut
   ON MISSION(statut_mission);
```

La clé primaire pk_mission identifie chaque mission. La contrainte ck_mission_dates garantit qu'une date de fin, si elle existe, ne soit jamais antérieure à la date de début. Le CHECK sur statut_mission borne les valeurs aux états attendus.

Côté indexation, la PK indexe id_mission. L'index idx_mission_statut est pertinent si vous filtrez souvent les missions actives ou terminées, ce qui est probable dans les requêtes de suivi.

FENETRE_COM

```
CREATE TABLE FENETRE_COM(
   id_fenetre NUMBER(10),
   datetime_debut TIMESTAMP NOT NULL,
   duree NUMBER(4,0) NOT NULL,
   elevation_max NUMBER(5,2) NOT NULL,
   volume_donnees NUMBER(8,1),
   statut VARCHAR2(20) NOT NULL,
   code_station VARCHAR2(20) NOT NULL,
   id_satellite VARCHAR2(20) NOT NULL,
   CONSTRAINT pk_fenetre_com PRIMARY KEY(id_fenetre),
   CONSTRAINT fk_fenetre_station FOREIGN KEY(code_station) REFERENCES
STATION_SOL(code_station),
   CONSTRAINT fk_fenetre_satellite FOREIGN KEY(id_satellite) REFERENCES
SATELLITE(id_satellite),
   CONSTRAINT ck_fenetre_duree CHECK (duree BETWEEN 1 AND 900),
   CONSTRAINT ck_fenetre_elevation CHECK (elevation_max BETWEEN 0 AND
90),
   CONSTRAINT ck_fenetre_statut CHECK (statut IN ('Planifiée',
'Réalisée')),
   CONSTRAINT ck_fenetre_volume CHECK (volume_donnees IS NULL OR
volume_donnees >= 0)
);

CREATE INDEX idx_fenetre_code_station
   ON FENETRE_COM(code_station);

CREATE INDEX idx_fenetre_id_satellite
   ON FENETRE_COM(id_satellite);

CREATE INDEX idx_fenetre_datetime
   ON FENETRE_COM(datetime_debut);
```

La clé primaire pk_fenetre_com identifie chaque fenêtre. Les deux clés étrangères assurent qu'une fenêtre concerne obligatoirement une station et un satellite. La contrainte ck_fenetre_duree est explicitement demandée par le sujet avec une durée entre 1 et 900 secondes. Les autres CHECK encadrent l'élévation, le statut et le volume de données.

Côté indexation, la PK couvre id_fenetre. L'index idx_fenetre_code_station accélère les recherches de fenêtres par station. L'index idx_fenetre_id_satellite sert aux recherches par satellite. L'index idx_fenetre_datetime est utile pour les plannings chronologiques, les tris, et les vérifications de chevauchement avant mise en place des triggers métier.

HISTORIQUE_STATUT

```
CREATE TABLE HISTORIQUE_STATUT(
   id_historique NUMBER(10),
   statut VARCHAR2(30) NOT NULL,
   timestamp TIMESTAMP NOT NULL,
   id_satellite VARCHAR2(20) NOT NULL,
   CONSTRAINT pk_historique_statut PRIMARY KEY(id_historique),
   CONSTRAINT fk_hist_satellite FOREIGN KEY(id_satellite) REFERENCES
SATELLITE(id_satellite),
   CONSTRAINT ck_historique_statut CHECK (statut IN ('Opérationnel', 'En
veille', 'Défaillant', 'Désorbité'))
);

CREATE INDEX idx_historique_id_satellite
   ON HISTORIQUE_STATUT(id_satellite);

CREATE INDEX idx_historique_sat_ts
   ON HISTORIQUE_STATUT(id_satellite, timestamp);
```

La clé primaire pk_historique_statut identifie chaque événement d'historisation. La clé étrangère garantit le rattachement à un satellite existant. Le CHECK sur statut maintient l'historique dans le même domaine de valeurs que la table SATELLITE.

Côté indexation, la PK couvre id_historique. L'index idx_historique_id_satellite facilite la recherche de tout l'historique d'un satellite. L'index composite idx_historique_sat_ts optimise les requêtes chronologiques du type "dernier statut de SAT-003" ou "historique ordonné d'un satellite".

EMBARQUEMENT

```
CREATE TABLE EMBARQUEMENT(
   id_satellite VARCHAR2(20),
   ref_instrument VARCHAR2(20),
   date_integration DATE NOT NULL,
   etat_fonctionnement VARCHAR2(20) NOT NULL,
```

```
   commentaire CLOB,
   CONSTRAINT pk_embarquement PRIMARY KEY(id_satellite, ref_instrument),
   CONSTRAINT fk_emb_satellite FOREIGN KEY(id_satellite) REFERENCES
SATELLITE(id_satellite),
   CONSTRAINT fk_emb_instrument FOREIGN KEY(ref_instrument) REFERENCES
INSTRUMENT(ref_instrument),
   CONSTRAINT ck_emb_etat CHECK (etat_fonctionnement IN ('Nominal',
'Dégradé', 'Hors service'))
);

CREATE INDEX idx_emb_ref_instrument
   ON EMBARQUEMENT(ref_instrument);
```

La clé primaire composite pk_embarquement traduit bien l'association porteuse d'attributs entre SATELLITE et INSTRUMENT. Une même paire satellite-instrument ne peut donc apparaître qu'une seule fois. Le CHECK sur etat_fonctionnement encadre les états attendus de l'instrument embarqué.

Côté indexation, la PK crée un index sur (id_satellite, ref_instrument), ce qui suffit pour les recherches commençant par id_satellite. L'index supplémentaire idx_emb_ref_instrument est utile car la PK composite n'optimise pas bien les recherches faites uniquement par ref_instrument, par exemple "sur quels satellites est embarqué cet instrument ?".

AFFECTATION_STATION

```
CREATE TABLE AFFECTATION_STATION(
   id_centre VARCHAR2(20),
   code_station VARCHAR2(20),
   date_affectation DATE NOT NULL,
   commentaire CLOB,
   CONSTRAINT pk_affectation_station PRIMARY KEY(id_centre,
code_station),
   CONSTRAINT fk_aff_centre FOREIGN KEY(id_centre) REFERENCES
CENTRE_CONTROLE(id_centre),
   CONSTRAINT fk_aff_station FOREIGN KEY(code_station) REFERENCES
STATION_SOL(code_station)
);

CREATE INDEX idx_emb_ref_instrument
   ON EMBARQUEMENT(ref_instrument);
```

La clé primaire composite pk_affectation_station empêche de dupliquer une affectation entre un centre et une station. Les deux clés étrangères assurent l'intégrité entre référentiel des centres et référentiel des stations.

Côté indexation, la PK indexe (id_centre, code_station) et convient pour les recherches par centre. L'index idx_aff_code_station complète ce dispositif pour les requêtes inverses, par exemple "quel centre supervise cette station ?".

PARTICIPATION

```
CREATE TABLE PARTICIPATION(
   id_satellite VARCHAR2(20),
   id_mission VARCHAR2(20),
   role_satellite VARCHAR2(100) NOT NULL,
   commentaire CLOB,
   CONSTRAINT pk_participation PRIMARY KEY(id_satellite, id_mission),
   CONSTRAINT fk_part_satellite FOREIGN KEY(id_satellite) REFERENCES
SATELLITE(id_satellite),
   CONSTRAINT fk_part_mission FOREIGN KEY(id_mission) REFERENCES
MISSION(id_mission)
);

CREATE INDEX idx_part_id_mission
   ON PARTICIPATION(id_mission);
```

La clé primaire composite pk_participation formalise la relation plusieurs-à-plusieurs entre satellites et missions. Une même participation ne peut pas être dupliquée. Les clés étrangères garantissent que chaque lien associe un satellite et une mission existants.

Côté indexation, la PK indexe (id_satellite, id_mission) et favorise les recherches par satellite. L'index idx_part_id_mission est nécessaire pour accélérer l'autre sens de lecture, c'est-à-dire "quels satellites participent à cette mission ?".

Réponse aux questions :

Q1 — Pourquoi ne peut-on pas créer SATELLITE avant ORBITE ? Quelle règle de gestion cela traduit-il ?

On ne peut pas créer la table SATELLITE avant la table ORBITE, car SATELLITE contient une clé étrangère id_orbite qui référence la clé primaire de ORBITE. En Oracle, la table référencée doit exister avant la déclaration de la contrainte de clé étrangère. Cela traduit la règle de gestion selon laquelle chaque satellite est rattaché à une orbite courante, et qu'une même orbite peut être partagée par plusieurs satellites. Cette dépendance correspond à la relation métier décrite par RG-S02 et RG-O01.

Q2 — La règle RG-S06 (satellite désorbité : plus de fenêtre ni de mission) peut-elle être vérifiée au niveau DDL seul ? Quelle solution proposez-vous ?

Non, cette règle ne peut pas être vérifiée au niveau DDL seul. Une contrainte DDL classique comme CHECK, NOT NULL, UNIQUE ou FOREIGN KEY ne permet pas de contrôler une

condition dépendant d'une autre table. Le cahier des charges précise d'ailleurs explicitement que RG-S06 ne peut pas être vérifiée au niveau DDL seul. La bonne solution est donc d'utiliser un trigger BEFORE INSERT OR UPDATE sur FENETRE_COM et sur PARTICIPATION, qui interroge la table SATELLITE et bloque l'opération si le satellite est au statut Désorbité.

Q3 — Comment implémentez-vous la contrainte RG-F02 (pas de chevauchement de fenêtres pour un même satellite) ? Cette contrainte est-elle directement exprimable en CHECK ?

Cette contrainte ne peut pas être exprimée directement avec un CHECK, car un CHECK ne compare que les valeurs de la ligne courante et ne peut pas vérifier l'existence d'un recouvrement avec d'autres lignes de la même table. Pour implémenter RG-F02, il faut un trigger BEFORE INSERT OR UPDATE sur FENETRE_COM. Ce trigger doit calculer l'intervalle temporel de la nouvelle fenêtre à partir de datetime_debut et de duree, puis rechercher dans FENETRE_COM une autre ligne concernant le même satellite dont l'intervalle se chevauche avec celui de la nouvelle ligne. Si un conflit existe, le trigger lève une erreur avec RAISE_APPLICATION_ERROR.

Q4 — Quel type Oracle choisissez-vous pour format_cubesat (valeurs possibles : 1U, 3U, 6U, 12U) ? Justifiez.

Le type le plus adapté est VARCHAR2(5). Les valeurs possibles 1U, 3U, 6U et 12U sont des codes alphanumériques, pas des nombres purs, puisqu'elles combinent une partie numérique et la lettre U. Un type numérique ne conviendrait donc pas. VARCHAR2(5) est suffisant pour stocker toutes les valeurs prévues, et il est cohérent avec le dictionnaire des données du projet. Pour sécuriser le domaine, on ajoute un CHECK du type format_cubesat IN ('1U','3U','6U','12U').

Partie 2 — Jeu de données initial (DML)

Pour cette partie, un simple script d'insert suffit :

```
/* 1) ORBITE */
INSERT INTO ORBITE (
    id_orbite, type_orbite, altitude, inclinaison, periode_orbitale,
excentricite, zone_couverture
) VALUES (
    'ORB-001', 'SSO', 550, 97.6, 95.5, 0.0010, 'Polaire globale --
Europe / Arctique'
);

INSERT INTO ORBITE (
    id_orbite, type_orbite, altitude, inclinaison, periode_orbitale,
excentricite, zone_couverture
```

```sql
) VALUES (
    'ORB-002', 'SSO', 700, 98.2, 98.8, 0.0008, 'Polaire globale -- haute
latitude'
);

INSERT INTO ORBITE (
    id_orbite, type_orbite, altitude, inclinaison, periode_orbitale,
excentricite, zone_couverture
) VALUES (
    'ORB-003', 'LEO', 400, 51.6, 92.6, 0.0020, 'Équatoriale -- zone
tropicale'
);

/* 2) SATELLITE */
INSERT INTO SATELLITE (
    id_satellite, nom_satellite, date_lancement, masse, format_cubesat,
statut,
    duree_vie_prevue, capacite_batterie, id_orbite
) VALUES (
    'SAT-001', 'NanoOrbit-Alpha', DATE '2022-03-15', 1.30, '3U',
'Opérationnel',
    60, 20, 'ORB-001'
);

INSERT INTO SATELLITE (
    id_satellite, nom_satellite, date_lancement, masse, format_cubesat,
statut,
    duree_vie_prevue, capacite_batterie, id_orbite
) VALUES (
    'SAT-002', 'NanoOrbit-Beta', DATE '2022-03-15', 1.30, '3U',
'Opérationnel',
    60, 20, 'ORB-001'
);

INSERT INTO SATELLITE (
    id_satellite, nom_satellite, date_lancement, masse, format_cubesat,
statut,
    duree_vie_prevue, capacite_batterie, id_orbite
) VALUES (
    'SAT-003', 'NanoOrbit-Gamma', DATE '2023-06-10', 2.00, '6U',
'Opérationnel',
    84, 40, 'ORB-002'
);
```

```sql
INSERT INTO SATELLITE (
    id_satellite, nom_satellite, date_lancement, masse, format_cubesat,
statut,
    duree_vie_prevue, capacite_batterie, id_orbite
) VALUES (
    'SAT-004', 'NanoOrbit-Delta', DATE '2023-06-10', 2.00, '6U', 'En
veille',
    84, 40, 'ORB-002'
);

INSERT INTO SATELLITE (
    id_satellite, nom_satellite, date_lancement, masse, format_cubesat,
statut,
    duree_vie_prevue, capacite_batterie, id_orbite
) VALUES (
    'SAT-005', 'NanoOrbit-Epsilon', DATE '2021-11-20', 4.50, '12U',
'Désorbité',
    36, 80, 'ORB-003'
);

/* 3) INSTRUMENT */
INSERT INTO INSTRUMENT (
    ref_instrument, type_instrument, modele, resolution, consommation,
masse
) VALUES (
    'INS-CAM-01', 'Caméra optique', 'PlanetScope-Mini', 3, 2.5, 0.40
);

INSERT INTO INSTRUMENT (
    ref_instrument, type_instrument, modele, resolution, consommation,
masse
) VALUES (
    'INS-IR-01', 'Infrarouge', 'FLIR-Lepton-3', 160, 1.2, 0.15
);

INSERT INTO INSTRUMENT (
    ref_instrument, type_instrument, modele, resolution, consommation,
masse
) VALUES (
    'INS-AIS-01', 'Récepteur AIS', 'ShipTrack-V2', NULL, 0.8, 0.12
);

INSERT INTO INSTRUMENT (
    ref_instrument, type_instrument, modele, resolution, consommation,
```

```sql
masse
) VALUES (
    'INS-SPEC-01', 'Spectromètre', 'HyperSpec-Nano', 30, 3.1, 0.60
);

/* 4) EMBARQUEMENT */
INSERT INTO EMBARQUEMENT (
    id_satellite, ref_instrument, date_integration, etat_fonctionnement,
commentaire
) VALUES (
    'SAT-001', 'INS-CAM-01', DATE '2022-03-15', 'Nominal',
    'Imageur principal Alpha -- utilisé dans MSN-DEF-2022 et
MSN-ARC-2023'
);

INSERT INTO EMBARQUEMENT (
    id_satellite, ref_instrument, date_integration, etat_fonctionnement,
commentaire
) VALUES (
    'SAT-001', 'INS-IR-01', DATE '2022-03-15', 'Nominal',
    'Détection thermique complémentaire'
);

INSERT INTO EMBARQUEMENT (
    id_satellite, ref_instrument, date_integration, etat_fonctionnement,
commentaire
) VALUES (
    'SAT-002', 'INS-CAM-01', DATE '2022-03-15', 'Nominal',
    'Imageur secondaire -- même modèle que SAT-001 (achat en lot)'
);

INSERT INTO EMBARQUEMENT (
    id_satellite, ref_instrument, date_integration, etat_fonctionnement,
commentaire
) VALUES (
    'SAT-003', 'INS-CAM-01', DATE '2023-06-10', 'Nominal',
    'Caméra haute résolution -- 6U offre plus d''espace'
);

INSERT INTO EMBARQUEMENT (
    id_satellite, ref_instrument, date_integration, etat_fonctionnement,
commentaire
) VALUES (
    'SAT-003', 'INS-SPEC-01', DATE '2023-06-10', 'Nominal',
```

```sql
    'Spectromètre -- mission surveillance qualité de l''air'
);

INSERT INTO EMBARQUEMENT (
    id_satellite, ref_instrument, date_integration, etat_fonctionnement,
commentaire
) VALUES (
    'SAT-004', 'INS-IR-01', DATE '2023-06-10', 'Dégradé',
    'Résolution réduite -- satellite en veille depuis anomalie
thermique'
);

INSERT INTO EMBARQUEMENT (
    id_satellite, ref_instrument, date_integration, etat_fonctionnement,
commentaire
) VALUES (
    'SAT-005', 'INS-AIS-01', DATE '2021-11-20', 'Hors service',
    'SAT-005 désorbité -- instrument non récupérable'
);

/* 5) CENTRE_CONTROLE */
INSERT INTO CENTRE_CONTROLE (
    id_centre, nom_centre, ville, region_geo, fuseau_horaire, statut
) VALUES (
    'CTR-001', 'NanoOrbit Paris HQ', 'Paris', 'Europe', 'Europe/Paris',
'Actif'
);

INSERT INTO CENTRE_CONTROLE (
    id_centre, nom_centre, ville, region_geo, fuseau_horaire, statut
) VALUES (
    'CTR-002', 'NanoOrbit Houston', 'Houston', 'Amériques',
'America/Chicago', 'Actif'
);

/* 6) STATION_SOL */
INSERT INTO STATION_SOL (
    code_station, nom_station, latitude, longitude, diametre_antenne,
bande_frequence, debit_max, statut
) VALUES (
    'GS-TLS-01', 'Toulouse Ground Station', 43.6047, 1.4442, 3.5, 'S',
150, 'Active'
);
```

```sql
INSERT INTO STATION_SOL (
    code_station, nom_station, latitude, longitude, diametre_antenne,
bande_frequence, debit_max, statut
) VALUES (
    'GS-KIR-01', 'Kiruna Arctic Station', 67.8557, 20.2253, 5.4, 'X',
400, 'Active'
);

INSERT INTO STATION_SOL (
    code_station, nom_station, latitude, longitude, diametre_antenne,
bande_frequence, debit_max, statut
) VALUES (
    'GS-SGP-01', 'Singapore Station', 1.3521, 103.8198, 3.0, 'S', 120,
'Maintenance'
);

/* 7) AFFECTATION_STATION */
INSERT INTO AFFECTATION_STATION (
    id_centre, code_station, date_affectation, commentaire
) VALUES (
    'CTR-001', 'GS-TLS-01', DATE '2022-01-10',
    'Paris HQ supervise la station de Toulouse -- proximité
géographique'
);

INSERT INTO AFFECTATION_STATION (
    id_centre, code_station, date_affectation, commentaire
) VALUES (
    'CTR-001', 'GS-KIR-01', DATE '2022-01-10',
    'Paris HQ supervise également Kiruna -- missions polaires SSO'
);

INSERT INTO AFFECTATION_STATION (
    id_centre, code_station, date_affectation, commentaire
) VALUES (
    'CTR-002', 'GS-SGP-01', DATE '2023-03-15',
    'Houston supervise Singapour -- couverture zone Asie-Pacifique'
);

/* 8) MISSION */
INSERT INTO MISSION (
    id_mission, nom_mission, objectif, zone_geo_cible, date_debut,
date_fin, statut_mission
) VALUES (
```

```sql
    'MSN-ARC-2023', 'ArcticWatch 2023',
    'Surveillance fonte des glaces et dynamique des banquises',
    'Arctique / Groenland',
    DATE '2023-01-01', NULL, 'Active'
);

INSERT INTO MISSION (
    id_mission, nom_mission, objectif, zone_geo_cible, date_debut,
date_fin, statut_mission
) VALUES (
    'MSN-DEF-2022', 'DeforestAlert',
    'Détection et cartographie de la déforestation en temps quasi-réel',
    'Amazonie / Congo',
    DATE '2022-06-01', DATE '2023-05-31', 'Terminée'
);

INSERT INTO MISSION (
    id_mission, nom_mission, objectif, zone_geo_cible, date_debut,
date_fin, statut_mission
) VALUES (
    'MSN-COAST-2024', 'CoastGuard 2024',
    'Surveillance évolution du trait de côte et détection d''érosion',
    'Méditerranée / Atlantique',
    DATE '2024-03-01', NULL, 'Active'
);

/* 9) FENETRE_COM */

INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max, volume_donnees,
statut, code_station, id_satellite
) VALUES (
    1, TIMESTAMP '2024-01-15 09:14:00', 420, 82.3, 1250, 'Réalisée',
'GS-KIR-01', 'SAT-001'
);

INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max, volume_donnees,
statut, code_station, id_satellite
) VALUES (
    2, TIMESTAMP '2024-01-15 11:52:00', 310, 67.1, 890, 'Réalisée',
'GS-TLS-01', 'SAT-002'
);
```

```sql
INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max, volume_donnees,
statut, code_station, id_satellite
) VALUES (
    3, TIMESTAMP '2024-01-16 08:30:00', 540, 88.9, 1680, 'Réalisée',
'GS-KIR-01', 'SAT-003'
);

INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max, volume_donnees,
statut, code_station, id_satellite
) VALUES (
    4, TIMESTAMP '2024-01-20 14:22:00', 380, 71.4, NULL, 'Planifiée',
'GS-TLS-01', 'SAT-001'
);

INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max, volume_donnees,
statut, code_station, id_satellite
) VALUES (
    5, TIMESTAMP '2024-01-21 07:45:00', 290, 59.8, NULL, 'Planifiée',
'GS-TLS-01', 'SAT-003'
);

/* 10) PARTICIPATION */
INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-001', 'MSN-ARC-2023', 'Imageur principal',
    'Alpha produit les clichés haute résolution de la calotte glaciaire'
);

INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-002', 'MSN-ARC-2023', 'Imageur secondaire',
    'Beta couvre les zones d'ombre de Alpha (passages décalés de 45
min)'
);

INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-003', 'MSN-ARC-2023', 'Satellite de relais',
```

```
    'Gamma relaie les données vers GS-KIR-01 quand Alpha/Beta ne sont
pas en vue'
);

INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-001', 'MSN-DEF-2022', 'Imageur principal',
    'Mission terminée -- participation historique valide (date_fin =
2023-05-31)'
);

INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-005', 'MSN-DEF-2022', 'Imageur secondaire',
    'SAT-005 était opérationnel lors de la mission -- désorbité après la
fin de MSN-DEF-2022'
);

INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-003', 'MSN-COAST-2024', 'Imageur principal',
    'Gamma + spectromètre -- idéal pour la détection d''érosion côtière'
);

INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-004', 'MSN-COAST-2024', 'Satellite de secours',
    'Delta en veille -- activable si Gamma connaît une anomalie'
);

/* 11) HISTORIQUE_STATUT */
/* Aucun INSERT manuel : table alimentée par trigger */

COMMIT;
```

Partie 3 — Les 5 triggers métier NanoOrbit

T1 — trg_valider_fenetre

```
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE TRIGGER trg_valider_fenetre
BEFORE INSERT ON FENETRE_COM
FOR EACH ROW
DECLARE
v_statut_satellite SATELLITE.statut%TYPE;
v_statut_station STATION_SOL.statut%TYPE;
BEGIN
SELECT statut
INTO v_statut_satellite
FROM SATELLITE
WHERE id_satellite = :NEW.id_satellite;

IF v_statut_satellite = 'Désorbité' THEN
RAISE_APPLICATION_ERROR(
-20001,
'Insertion refusée : un satellite désorbité ne peut pas recevoir de
nouvelle fenêtre.'
);
END IF;

SELECT statut
INTO v_statut_station
FROM STATION_SOL
WHERE code_station = :NEW.code_station;

IF v_statut_station = 'Maintenance' THEN
RAISE_APPLICATION_ERROR(
-20002,
'Insertion refusée : une station en maintenance ne peut pas planifier de
nouvelle fenêtre.'
);
END IF;
END;
/
SHOW ERRORS;
```

Ce trigger se déclenche avant chaque insertion dans FENETRE_COM. Il vérifie deux règles du sujet :

- un satellite au statut Désorbité ne peut plus recevoir de nouvelle fenêtre ;
- une station au statut Maintenance ne peut pas planifier de nouvelle fenêtre.

T2 — trg_no_chevauchement

```
SET SERVEROUTPUT ON;
```

```oracle_plsql
CREATE OR REPLACE TRIGGER trg_no_chevauchement
BEFORE INSERT OR UPDATE ON FENETRE_COM
FOR EACH ROW
DECLARE
    v_nb_conflits_satellite NUMBER;
    v_nb_conflits_station   NUMBER;
    v_fin_nouvelle          TIMESTAMP;
BEGIN
    v_fin_nouvelle := :NEW.datetime_debut + NUMTODSINTERVAL(:NEW.duree,
'SECOND');

    SELECT COUNT(*)
    INTO v_nb_conflits_satellite
    FROM FENETRE_COM f
    WHERE f.id_satellite = :NEW.id_satellite
      AND (
            :NEW.id_fenetre IS NULL
            OR f.id_fenetre <> :NEW.id_fenetre
          )
      AND :NEW.datetime_debut < f.datetime_debut +
NUMTODSINTERVAL(f.duree, 'SECOND')
      AND v_fin_nouvelle > f.datetime_debut;

    IF v_nb_conflits_satellite > 0 THEN
        RAISE_APPLICATION_ERROR(
            -20003,
            'Insertion/Mise à jour refusée : chevauchement détecté pour
ce satellite.'
        );
    END IF;

    SELECT COUNT(*)
    INTO v_nb_conflits_station
    FROM FENETRE_COM f
    WHERE f.code_station = :NEW.code_station
      AND (
            :NEW.id_fenetre IS NULL
            OR f.id_fenetre <> :NEW.id_fenetre
          )
      AND :NEW.datetime_debut < f.datetime_debut +
NUMTODSINTERVAL(f.duree, 'SECOND')
      AND v_fin_nouvelle > f.datetime_debut;

    IF v_nb_conflits_station > 0 THEN
```

```
        RAISE_APPLICATION_ERROR(
            -20004,
            'Insertion/Mise à jour refusée : chevauchement détecté pour
cette station.'
        );
    END IF;
END;
/
SHOW ERRORS;
```

Ce trigger se déclenche avant insertion ou mise à jour d'une fenêtre. Il calcule l'intervalle de la nouvelle fenêtre à partir de datetime_debut et de duree, puis cherche s'il existe une autre fenêtre qui chevauche :

- le même satellite ;
- ou la même station.

T3 — trg_volume_realise

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE TRIGGER trg_volume_realise
BEFORE INSERT OR UPDATE ON FENETRE_COM
FOR EACH ROW
BEGIN
    IF :NEW.statut <> 'Réalisée' THEN
        :NEW.volume_donnees := NULL;
    END IF;
END;
/
SHOW ERRORS;
```

Ce trigger garantit la règle RG-F05 : le volume téléchargé ne peut être renseigné que si la fenêtre est Réalisée. Si le statut est différent, le trigger force volume_donnees à NULL.

T4 — trg_mission_terminee

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE TRIGGER trg_mission_terminee
BEFORE INSERT ON PARTICIPATION
FOR EACH ROW
DECLARE
    v_statut_mission MISSION.statut_mission%TYPE;
BEGIN
    SELECT statut_mission
    INTO v_statut_mission
    FROM MISSION
```

```
    WHERE id_mission = :NEW.id_mission;

    IF v_statut_mission = 'Terminée' THEN
        RAISE_APPLICATION_ERROR(
            -20005,
            'Insertion refusée : impossible d''ajouter un satellite à
une mission terminée.'
        );
    END IF;
END;
/
SHOW ERRORS;
```

Ce trigger se déclenche avant insertion dans PARTICIPATION. Il empêche l'ajout d'un satellite à une mission déjà terminée.

T5 — trg_historique_statut

SET SERVEROUTPUT ON;

Le sujet demande un trigger AFTER UPDATE OF statut ON SATELLITE pour assurer la traçabilité des changements de statut. Dans l'énoncé officiel, la table d'historique contient l'ancien statut et le nouveau. Chez nous, ce n'est pas le cas : on a choisi une table plus simple, avec une seule colonne statut et un horodatage.

Notre trigger T5 a donc été adapté pour insérer uniquement le nouveau statut à chaque changement réel. Ce trigger nécessite l'implémentation d'une séquence pour l'auto-incrémentation de l'id :

```
CREATE SEQUENCE seq_historique_statut
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

Partie 4 — Script de contrôle

Test T1 :

```
-- Cas valide : insertion d'une fenêtre pour un satellite opérationnel
-- et une station active
INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max,
    volume_donnees, statut, code_station, id_satellite
) VALUES (
```

```sql
    100, TIMESTAMP '2024-01-22 10:00:00', 300, 70.0,
    NULL, 'Planifiée', 'GS-TLS-01', 'SAT-002'
);

-- Cas en erreur : satellite désorbité
INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max,
    volume_donnees, statut, code_station, id_satellite
) VALUES (
    101, TIMESTAMP '2024-01-22 11:00:00', 300, 70.0,
    NULL, 'Planifiée', 'GS-TLS-01', 'SAT-005'
);

-- Cas en erreur : station en maintenance
INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max,
    volume_donnees, statut, code_station, id_satellite
) VALUES (
    102, TIMESTAMP '2024-01-22 12:00:00', 300, 70.0,
    NULL, 'Planifiée', 'GS-SGP-01', 'SAT-001'
);
```

Résultats :

Pour le premier, le résultat est bon : 1 row created.

Pour le second, le résultat est bon (erreur) :

```
ERROR at line 1:
ORA-20001: Insertion refusée : un satellite désorbité ne peut pas
recevoir de
nouvelle fenêtre.
ORA-06512: at "NANO_ADMIN.TRG_VALIDER_FENETRE", line 11
ORA-04088: error during execution of trigger
'NANO_ADMIN.TRG_VALIDER_FENETRE'
```

Et pour le 3ème, le résultat est également bon (erreur) :

```
ERROR at line 1:
ORA-20002: Insertion refusée : une station en maintenance ne peut pas
planifier
de nouvelle fenêtre.
ORA-06512: at "NANO_ADMIN.TRG_VALIDER_FENETRE", line 23
ORA-04088: error during execution of trigger
'NANO_ADMIN.TRG_VALIDER_FENETRE'
```

Test T2 :

```
-- Cas en erreur : chevauchement pour SAT-001
-- SAT-001 a déjà une fenêtre le 2024-01-20 à 14:22:00 sur 380 secondes
INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max,
    volume_donnees, statut, code_station, id_satellite
) VALUES (
    103, TIMESTAMP '2024-01-20 14:25:00', 200, 65.0,
    NULL, 'Planifiée', 'GS-KIR-01', 'SAT-001'
);

-- Cas en erreur : chevauchement pour GS-TLS-01
INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max,
    volume_donnees, statut, code_station, id_satellite
) VALUES (
    104, TIMESTAMP '2024-01-21 07:46:00', 120, 60.0,
    NULL, 'Planifiée', 'GS-TLS-01', 'SAT-002'
);
```

Résultats :

Pour le premier, le résultat est bon (erreur) :

```
ERROR at line 1:
ORA-20003: Insertion/Mise à jour refusée : chevauchement détecté pour ce
satellite.
ORA-06512: at "NANO_ADMIN.TRG_NO_CHEVAUCHEMENT", line 20
ORA-04088: error during execution of trigger
'NANO_ADMIN.TRG_NO_CHEVAUCHEMENT'
```

Le second est bon également :

```
ERROR at line 1:
ORA-20004: Insertion/Mise à jour refusée : chevauchement détecté pour
cette station.
ORA-06512: at "NANO_ADMIN.TRG_NO_CHEVAUCHEMENT", line 38
ORA-04088: error during execution of trigger
'NANO_ADMIN.TRG_NO_CHEVAUCHEMENT'
```

Test T3 :

```
-- Cas valide : si statut = Planifiée, le volume est forcé à NULL
INSERT INTO FENETRE_COM (
    id_fenetre, datetime_debut, duree, elevation_max,
    volume_donnees, statut, code_station, id_satellite
```

```sql
) VALUES (
    105, TIMESTAMP '2024-01-23 09:00:00', 250, 55.0,
    999, 'Planifiée', 'GS-KIR-01', 'SAT-002'
);

SELECT id_fenetre, statut, volume_donnees
FROM FENETRE_COM
WHERE id_fenetre = 105;
```

Résultat : 1 row created

Le select renvoi :

```
ID_FENETRE STATUT       VOLUME_DONNEES
---------- -------------------- --------------
       105 Planifiée
```

Le volume est bien null donc la trigger fonctionne.

Test T4 :

```sql
-- Cas valide : ajout à une mission active
INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-002', 'MSN-COAST-2024', 'Satellite de relais',
    'Ajout valide à une mission active'
);

-- Cas en erreur : ajout à une mission terminée
INSERT INTO PARTICIPATION (
    id_satellite, id_mission, role_satellite, commentaire
) VALUES (
    'SAT-004', 'MSN-DEF-2022', 'Satellite de secours',
    'Test refus mission terminée'
);
```

Résultats :

Pour le premier, le résultat est bon, nous avons : 1 row created.

Et pour le second, nous avons bien une erreur :

```
ERROR at line 1:
ORA-20005: Insertion refusée : impossible d'ajouter un satellite à une
```

```
mission terminée.
ORA-06512: at "NANO_ADMIN.TRG_MISSION_TERMINEE", line 10
ORA-04088: error during execution of trigger
'NANO_ADMIN.TRG_MISSION_TERMINEE'
```

Test T5 :

```
-- Cas valide : changement de statut, création automatique d'une ligne
UPDATE SATELLITE
SET statut = 'Défaillant'
WHERE id_satellite = 'SAT-004';

SELECT *
FROM HISTORIQUE_STATUT
WHERE id_satellite = 'SAT-004'
ORDER BY timestamp_;

COMMIT;
```

Et pour ce dernier test, nous faisons un update sans soucis : 1 row updated.

Et le résultat du select est sans appel :
ID_HISTORIQUE = 1
STATUT = Défaillant
TIMESTAMP = 03-APR-26 12.39.03.931985 PM
ID_SATELLITE = SAT-004

Partie 5 — Triggers et séquence pour la génération automatique.

Pour la génération automatiques d'identifiants ne comprenant pas de références métiers comme SATELLITE, ORBITE, CENTRE_CONTROLE et FENETRE_COM. Pour la table historique du statut, cette logique à déjà été implémentée plus tôt.

Pour chaque table concernée, nous allons donc faire une séquence ainsi qu'un trigger à l'insertion qui va automatiquement choisir le nouvel identifiant.

SATELLITE :

```
CREATE SEQUENCE seq_satellite
START WITH 6
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

```
CREATE OR REPLACE TRIGGER trg_satellite_id
BEFORE INSERT ON SATELLITE
FOR EACH ROW
BEGIN
    IF :NEW.id_satellite IS NULL THEN
        :NEW.id_satellite := 'SAT-' || LPAD(seq_satellite.NEXTVAL, 3,
'0');
    END IF;
END;
/
```

ORBITE :

```
CREATE SEQUENCE seq_orbite
START WITH 4
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE OR REPLACE TRIGGER trg_orbite_id
BEFORE INSERT ON ORBITE
FOR EACH ROW
BEGIN
    IF :NEW.id_orbite IS NULL THEN
        :NEW.id_orbite := 'ORB-' || LPAD(seq_orbite.NEXTVAL, 3, '0');
    END IF;
END;
/
```

CENTRE_CONTROLE :

```
CREATE SEQUENCE seq_centre
START WITH 3
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE OR REPLACE TRIGGER trg_centre_id
BEFORE INSERT ON CENTRE_CONTROLE
FOR EACH ROW
BEGIN
    IF :NEW.id_centre IS NULL THEN
```

```
        :NEW.id_centre := 'CTR-' || LPAD(seq_centre.NEXTVAL, 3, '0');
    END IF;
END;
/
```

FENETRE_COM

```
CREATE SEQUENCE seq_fenetre
START WITH 6
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE OR REPLACE TRIGGER trg_fenetre_id
BEFORE INSERT ON FENETRE_COM
FOR EACH ROW
BEGIN
    IF :NEW.id_fenetre IS NULL THEN
        :NEW.id_fenetre := seq_fenetre.NEXTVAL;
    END IF;
END;
/
```